COMP1870

Graph Theory

Theoretical Foundations Of Computer Science

Fahad Panolan

School of Computer Science, University of Leeds, UK

Contents

1 Graph Theory – Basics **4**
1.1 Definitions: Graphs, multigraphs, and digraphs 5
1.2 Graph terminology 8
1.3 Degree 8
1.4 Isomorphism 10
1.5 Subgraphs 11
1.6 Walks, Paths, and Cycles 12
1.7 Special graphs 14

2 Representation of graphs **15**

3 Computation on graphs **18**
3.1 Number of Walks 18
3.2 Graph Isomorphism 21

4 Connectivity **21**
4.1 Connected components 21
4.2 Algorithm: Connectivity 22

5 Euler's Theorem **23**
5.1 Eulerian tail and tour 23
5.2 Recognition algorithm for Eulerian graphs 24

6 Trees **25**
6.1 Characterization of Trees 27
6.2 Spanning Trees 29

7 Bipartite Matching and Organ Donation **30**
7.1 Matching 30
7.2 Algorithm for Bipartite Maximum Matching 31
7.3 Kidney Exchange 35

Acknowledgement

Following materials are referred for making this lecture note.

[1] Lecture Note *Graph Theory* by Benny Sudakov.

[2] Lecture Note *Introduction to Discrete Mathematics* by Kristina Vuskovic.

[3] Lecture Note *Algorithmic Game Theory* by Tim Roughgarden.

[4] Book *Introduction to Graph Theory* by Douglas West.

[5] Book *Graph Theory* by Reinhard Diestel.

1 Graph Theory – Basics

Recall that you have already seen graphs informally in this module. In the first lecture, you have seen bipartite graphs and an algorithm for finding maximum matching on bipartite graphs.

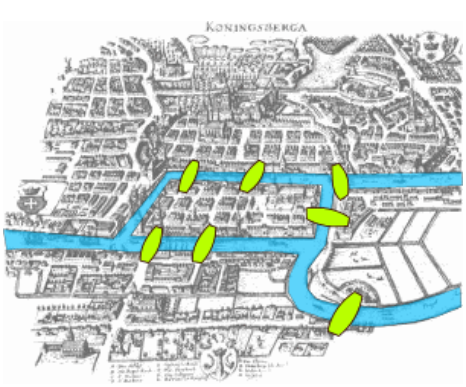
Graphs are mathematical structures used to model pairwise relationship between objects. Objects (called *vertices* or *nodes*) represents items of interests such as programs or tasks, computers or processing units, web pages, cities, peoples etc. There is an *edge* between two nodes if they are related in a certain way. This relation can be directed (one way relation) or undirected (symmetric relation). For example in a social network, we use nodes to represent people and an edge between two persons if they are friends. In this case the relation is symmetric. There are cases where the edge denotes a one-way relationship, for example, where one web page points to the other if there is a link to the latter web page in the former web page.

In graph theory, we study graphs, properties of graphs, modelling our problems or task as graphs and computation on graphs. The topics you have learned in this module – sets, functions, relations, logic and proofs – are prerequisites for learning graph theory. We mostly consider undirected graphs (i.e., the relation between nodes is symmetric).

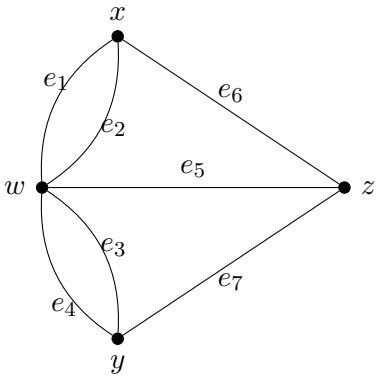
The Königsberg Bridge problem is a historically notable problem in mathematics and its resolution by Leonhard Euler in 1736, laid the foundations of graph theory.

The Königsberg Bridge problem: The city of Königsberg in Prussia was located on both sides of the Pregel river, and included two large islands (named Kneiphof and Lomse), and two mainland portions of the city, and the land masses are connected by seven bridges (See Figure 1a). The problem was to devise a walk through the city that would cross each of those bridges exactly once and return to the starting point. The problem reduces to traversing the Figure 3a, where the dots represent land masses and curves represent bridges.

The model in Figure 3a makes it easy to argue that the desired traversal does not exits. Each time we enter and leave a land mass we use two bridges ending it. At the starting land mass, we use a bridge to start the traversal and a bridge to end the traversal; we can pair those two as well. As a result, the existence of the desired traversal is not possible, if there is a land mass with an odd number of edges ending there. Thus, the desired traversal does not exist in Königsberg.



(a) City of Königsberg



(b) Model as a multigraph

1.1 Definitions: Graphs, multigraphs, and digraphs

Definition 1.1. A multigraph G is a triple consisting of a **vertex set** $V(G)$, an **edge set** $E(G)$, and a function that maps each edge to two vertices (not necessarily distinct) called the **endpoints** of the edge.

The usual way to picture a multigraph is by drawing a dot for each vertex and a line or curve for each edge joining the dots corresponding to its endpoints. The Figure 3a is a pictorial representation of a multigraph G with vertex set $V(G) = \{w, x, y, z\}$, edge set $E(G) = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$, and the assignment of endpoints with the edges can be read from the figure. **Multiple edges** are the edges with same pair of endpoints. For example, e_1 and e_2 are multiple edges in the graph G.

In a multigraph, we can have edges with only one endpoint. Such edges are called **loops**. The edge e_5 is a loop in the graph H, given in Figure 2.

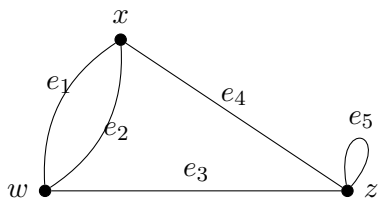


Figure 2: Multigraph H with loops

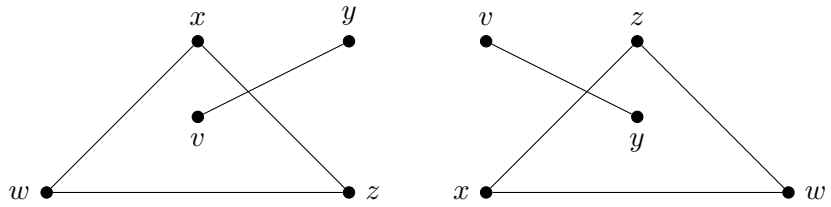
A **graph** is a multigraph without loops and multiple edges. Since there are no multiple edges, each edge in a graph can be uniquely represented by its endpoints. Thus we can ignore the relation associating edges with its

endpoints and define edges as a set of two element subsets of vertices.

Notation: For a set A, we use $\binom{A}{k}$ to denote the set of all k-element subsets of A.

Definition 1.2. A **graph** G consist of a set V (called **vertex set**) and a set E of two-element subsets of V. That is, $E \subseteq \binom{V}{2}$. E is called the **edge set** and the elements of E are called the **edges** of G. So, we write $G = (V, E)$ to represent the graph with vertex set V and edge set E. For a graph G, we also use $V(G)$ and $E(G)$ to denote its vertex set and edge set, respectively.

The graph $G = (\{v, w, x, y, z\}, \{\{v, y\}, \{w, x\}, \{x, z\}, \{w, z\}\})$ can be drawn in many ways. Two drawings of G can be found in Figure 3.



(a) Drawing 1 (b) Drawing 2

Figure 3: Two drawings of the graph G with vertex set $V(G) = \{v, w, x, y, z\}$ and edge set $E(G) = \{\{v, y\}, \{w, x\}, \{x, z\}, \{w, z\}\}$.

Notice that $\{w, x\}$ and $\{x, w\}$ are different representations of the same edge between vertices w and x.

Remark 1.3. Some text books uses the term *graphs* for multigraphs and *simple graphs* for multigraphs without loops and multiple edges. In this module, we use the term "graphs" to denote the graphs in Definition 1.2. Moreover, throughout this module we consider only finite graphs. That is, the vertex set is a finite set.

Note: Definitions and terminology in graph theory are different in different sources (books, articles, lecture notes, etc).

Definition 1.4. A **directed graph** or **digraph** D is a pair (V, A), where V is a set, called **vertex set** and A is a set of ordered pairs of vertices, called the **edge set** or **arc set**. For each arc $e = (u, v) \in A$, u is called the **trail** and v is called the **head** of arc e.

Example 1.5. $D = (\{v, w, x, y, z\}, \{(v, y), (z, x), (x, w)\})$ is an example of a digraph. A drawing of D can be found in Figure 4.

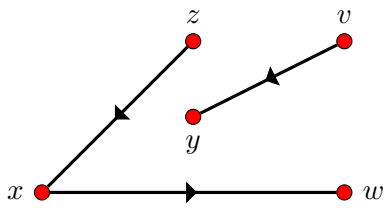


Figure 4: Drawing of a directed graph D with vertex set $V(D) = \{v, w, x, y, z\}$ and arc set $A(D) = \{(v, y), (z, x), (x, w)\}$.

In graphs, the edge relation is symmetric and in digraphs, an edge/arc denotes a one-way relationship. We use the term *graph* or *undirected graph* if the edge relation is symmetric and use the term *directed graph* or *digraph* if an edge denotes a one-way relationship. Some of the graph applications are listed in Table 1. In this table, identify the applications for which you use directed graphs.

Graph	**Nodes**	**Edges**
Transportation	street intersections	highways
Communication	computers	fiber optic cables
World Wide Web	web pages	hyperlinks
Social	people	relationships
Facebook	people	friendship
X (twitter)	people	following
Food web	species	predator-prey
Software systems	functions	function calls
Scheduling	tasks	precedence constraints
Circuits	gates	wires

Table 1: Graph Applications

Weighted Graphs. A weighted graph is a graph with a weight associated with each edge. Formally, a weighted graph consists of a graph G and weight function $w\colon E(G) \to \mathbb{R}$. The range of the weight function w can be natural numbers, integers, non-negative real numbers, rational numbers, etc, depending on the situation. An example of a weighted graph can be found in Figure 5.

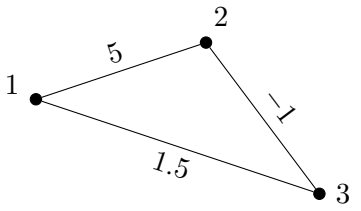


Figure 5: A weighted graph where the weight of $\{1, 2\}$ is 5

1.2 Graph terminology

In this module we focus mostly on graphs, that is without loops and multiple edges and directed edge. A graph with vertex set V is said to be a graph on V. Next, we introduce some terminology and notations related to graphs. Let G be a graph.

- We use $V(G)$ to denote its vertex set and $E(G)$ to denote its edge set.
- An edge $\{x, y\}$ may also be written as xy (or yx). For an edge $e = \{x, y\}$, x and y are the *endvertices* (or *endpoints*) of e.
- A vertex v is incident with an edge e if $v \in e$. We also say that e is an edge at v and e is incident to v.
- Two vertices x, y are *adjacent*, or *neighbours*, if $\{x, y\} \in E(G)$. Two distinct edges $e, f \in E(G)$ are *adjacent* if they have an endpoint in common (i.e., $e \cap f \neq \emptyset$).
- For a vertex v of a graph G, the set of neighbours of v is denoted by $N_G(v)$ or briefly $N(v)$. That is, $N_G(v) = \{u \in V(G) \mid uv \in E(G)\}$. We drop the subscript G from $N_G(v)$, if the graph G is clear from the context.

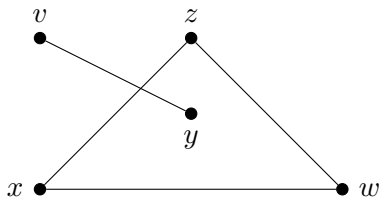
The above terminology can be extended to multigraphs.

1.3 Degree

Definition 1.6. Let G be a graph. The **degree** of a vertex $v \in V(G)$, denoted by $d_G(v)$ (or $d(v)$ if the graph is clear from the context) is the number of edges incident to v, which is also equal to the cardinality of the set $N(v)$.

The definition of degree in a multigraph is slightly different. The degree of a vertex v in a multigraph G is the number of edges incident to v, except that each loop at v counts twice. For example, the degree of z in the multigraph H in Figure 2, is 4.

Example 1.7. Degrees of the below graph: $d(v) = 1$, $d(w) = 2$, $d(x) = 2$, $d(y) = 1$, and $d(z) = 2$.



A degree sequence of a graph G is a sequence of the degree of its vertices in some order. For example $1, 2, 2, 2, 1$ is a degree sequence of the graph in Example 1.7.

Definition 1.8. Let d be a non-negative integer. A graph G is **d-regular**, if the degree of every vertex in G is d.

Question 1.9. Is there a 3-regular graph on 9 vertices?

Lemma 1.10. *Let G be a graph. Then, $\sum_{v \in V(G)} d(v) = 2 \cdot |E(G)|$.*

Proof. In the sum $\sum_{v \in V(G)} d(v)$, every edge $e = uv$ is counted twice; once from u and once from v. □

Theorem 1.11. *The number of vertices of odd degree in a graph is even.*

Proof. Let $x_1, \ldots, x_n$ be the vertices of a graph G with even degree and let $y_1, \ldots, y_m$ be the vertices of odd degree. Let

$$\begin{aligned} S &= d(x_1) + \ldots + d(x_n) \\ T &= d(y_1) + \ldots + d(y_m) \end{aligned}$$

By Lemma 1.10, $S + T$ is even. Since S is the sum of even numbers, S is even. So T must be even. Since T is the sum of m odd numbers, m must be even. □

Thus, the answer to Question 1.9 is "no".

1.4 Isomorphism

Question 1.12. Clearly, the two graphs given below are not same, because their vertex sets are different. But, are these graphs the "same" in some sense?



Definition 1.13. Let G_1 and G_2 be two graphs. An **isomorphism** from G_1 to G_2 is a bijection $f\colon V(G_1) \to V(G_2)$ such that $\{u, v\} \in E(G_1)$ if and only if $\{f(u), f(v)\} \in E(G_2)$. We say G_1 is **isomorphic** to G_2 if there is an isomorphism from G_1 to G_2.

Notice that if G_1 is isomorphic to G_2, then G_2 is isomorphic to G_1, because the inverse of the isomorphism from G_1 to G_2 is an isomorphism from G_2 to G_1. Thus, we can say G_1 **and** G_2 **are isomorphic** if G_1 is isomorphic to G_2.

Example 1.14. Consider Question 1.12.



The bijection $f\colon \{w, x, y, z\} \to \{a, b, c, d\}$ given by

$$f(w) = a, f(x) = d, f(y) = c, f(z) = b$$

is an isomorphism from the graph on the left to the graph on the right. The inverse function f^{-1} is an isomorphism from the graph on the right to the graph on the left.

Remark 1.15. Let R be the relation on the set of all graphs defined as follows: $(G_1, G_2) \in R$ if G_1 is isomorphic to G_2. Then R is an equivalence relation.

- Reflexive: Any graph is isomorphic to itself.

- Symmetric: If G_1 is isomorphic to G_2, then G_2 is isomorphic to G_1.
- Transitive: If G_1 is isomorphic to G_2 and G_2 is isomorphic to G_3, then G_1 is isomorphic to G_3.

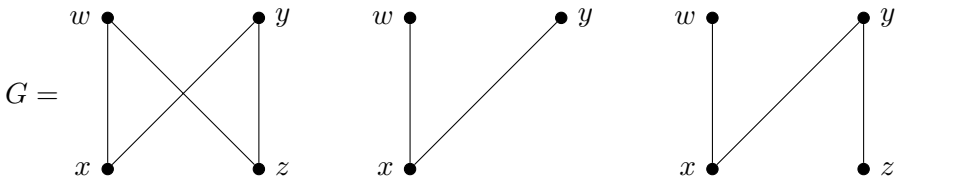
Exercise 1.16. Let R be the relation on the set of all graphs defined as follows: $(G_1, G_2) \in R$ if G_1 is isomorphic to G_2. Prove that R is an equivalence relation.

1.5 Subgraphs

Definition 1.17 (Subgraph)**.** Let $G = (V, E)$ and $G = (V', E')$ be two graphs such that $V' \subseteq V$ and $E' \subseteq E$. Then, G' is a **subgraph** of G (denoted by $G' \subseteq G$) and G is a **supergraph** of G'. If $G' \subseteq G$ and $V(G') = V(G)$, then G' is a **spanning subgraph** of G. If $G' \subseteq G$ and $G' \neq G$, then G' is a **proper subgraph** of G.

Definition 1.18 (Induced subgraph)**.** If $G' \subseteq G$, and G' contains all the edges $xy \in E(G)$ with $x, y \in V(G')$, then G' is an **induced subgraph** of G. For any vertex subset $U \subseteq V(G)$, we use $G[U]$ to denote the induced subgraph of G with vertex set U and edge set $\{xy \in E(G) \mid x, y \in U\}$. We also say that $G[U]$ is **the subgraph of G induced by U**.

Example 1.19. Example for subgraphs and induced subgraphs.



Induced subgraph

Spanning subgraph, but not induced subgraph

Definition 1.20. For a graph G and $S \subseteq V(G)$, $G - S$ is the subgraph obtained from G after deleting the vertices in S and the edges incident to vertices in S. In other words, $G - S = G[V(G) \setminus S]$. If $S = \{v\}$, then we write $G - v$ instead of $G - \{v\}$.

Definition 1.21. For a graph G and $F \subseteq E(G)$, $G - F$ is the spanning graph obtained from G after deleting the edges in F. That is, $V(G - F) = V(G)$ and $E(G - F) = E(G) \setminus F$. If $F = \{e\}$, then we write $G - e$ instead of $G - \{e\}$.

1.6 Walks, Paths, and Cycles

Definition 1.22. Let v_0 and v_k be vertices in a (multi)graph G. A **walk** W in G from v_0 to v_k of **length** k is an alternating sequence of $k+1$ vertices and k edges, $v_0, e_1, v_1, e_2, \ldots v_{k-1}, e_k, v_k$ in which edge e_i is incident on vertices v_{i-1} and v_i for every $i \in \{1, \ldots, k\}$. Here, v_0 is called the **start vertex** and v_k is called the **end vertex** of the walk W. Also, we say that W is a (v_0, v_k)-walk.

Note that in a graph, to denote a walk we just need to list the sequence of vertices since two vertices are connected by only one edge. For example $v_0, v_1 \ldots v_k$.

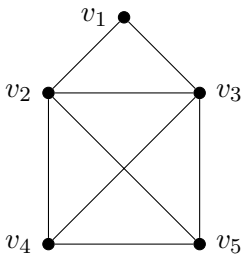
Definition 1.23. A walk in a (multi)graph G is called a **closed walk** if its start vertex and end vertex are same.

Definition 1.24. A walk (respectively, closed walk) is called a **trail** (respectively, **closed trail**) if all the edges in it are distinct.

Definition 1.25. A walk is called a **path** if all the vertices in it are distinct. If the start vertex of the path is u and the end vertex is v, then we say that it is a (u, v)-path.

Definition 1.26. A closed trail of non-zero length is called a **cycle** if all the vertices in it are distinct, except the start and the end vertex.

Example 1.27. Examples of walk, closed walk, path, cycle, trail and closed trail.



Walk:	$v_1, v_2, v_3, v_4, v_2, v_3$
Closed walk:	$v_1, v_2, v_3, v_4, v_2, v_3, v_1$
Path:	v_1, v_2, v_3, v_4
Cycle:	v_1, v_3, v_5, v_2, v_1
Trail:	v_2, v_1, v_3, v_2, v_4
Closed trail:	$v_2, v_1, v_3, v_2, v_4, v_5, v_2$

We say that a walk W **contains** a walk W' if the vertices and edges of W' occurs as a sublist of the vertices and edges of W, with the same order as in W' but not necessarily consecutive in W. In other words, consider W and W' as alternating sequences of vertices and edges. Then, W' is a subsequence of W (a subsequence of a given sequence is a sequence that

can be derived from the given sequence by deleting some or no elements without changing the order of the remaining elements). We would like to emphasise that the 'alternating sequence' for W' should be a subsequence of the 'alternating sequence' for W. In the case of graphs, when we consider W as a sequence of vertices, then some subsequence W' of W, may be a walk in the graph, but W **does not contain** W'. For example, consider the graph given in Example 1.27. The walk $W = v_2, v_3, v_1$ does not contain $W' = v_2, v_1$.

Every walk from u to v contains a path from u to v. This is sort of obvious. We prove this statement formally.

Lemma 1.28. *Let G be a graph and let $u, v \in V(G)$ be two distinct vertices. Every walk from u to v in G contains a path from u to v.*

We give two proofs for this lemma. First is a direct proof and the second is a proof by induction.

Proof 1: Let W be a walk from u to v in G. If W has no repeated vertices, then W is a path and we are done. So assume that W has a repeated vertex w. Let W' be the walk obtained by going along W from u to the first appearance of w, and then going along W from the last appearance of w to v. So W' is a walk from u to v whose length is smaller than the length of W. Moreover, W' is a subsequence of W. If W' has no repeated vertices, then we are done. Otherwise, repeat this procedure. By repeating this procedure until there are no more repeated vertices, we end up with a path from u to v. □

Proof 2: Proof by induction. Let $S(\ell)$ be the following statement. Every walk from u to v of length ℓ in G contains a path from u to v. We want to prove that for any positive integer ℓ, $S(\ell)$ is true.

We prove this statement by induction on ℓ.

Basis step: $\ell = 1$. Let W be a walk of length 1 from u to v. Then clearly, $W = u, uv, v$ and W is a path from u to v. So $S(1)$ is true.

Induction step: $\ell \geq 2$. We assume that $S(j)$ is true for all $j < \ell$.

Let $W = w_0, e_1, w_1, \ldots, e_\ell, w_\ell$ be a walk of length ℓ from u to v, where $u = w_0$ and $v = w_\ell$. If W has no repeated vertex, then W is path. Suppose W is not a path. Then, there exist $0 \leq i < j \leq \ell$ such that $w_i = w_j$.

Notice that $W' = w_0, e_1, \ldots, e_i, w_i, e_{j+1}, w_{j+1}, w_{j+2}, \ldots, w_\ell$ is a walk from u to v and W' is contained in W. Moreover, the length of W' is less than ℓ. Thus, by induction hypothesis W' contains a path P from u

to v. Since W contains W' and W' contains P, we get that W contains P. That is, we proved that $S(\ell)$ is true. □

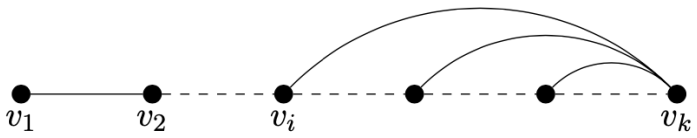
Exercise 1.29. Prove that any odd length closed walk in a graph G, contains a cycle in G. [**Hint**: A closed walk of length three in a graph G is a cycle.]

The **minimum degree** of a graph G is the minimum number among $\{d(v) \mid v \in V(G)\}$. For example the minimum degree of the graph in Example 1.27 is 2.

Theorem 1.30. *Every graph G with minimum degree $\delta \geq 2$ contains a path of length δ and a cycle of length at least $\delta + 1$.*

Proof. First we prove the statement that every graph G with minimum degree $\delta \geq 2$ contains a path of length δ. Let $P = v_1, \ldots, v_k$ be a longest path in G. Since P is a longest path, $N_G(v_k) \subseteq \{v_1, \ldots, v_{k-1}\}$. Also, since the minimum degree is δ, we get that $\delta \leq |N_G(v_k)| \leq k - 1$. Thus, the length of the path $k - 1 \geq \delta$.

Next we prove the statement that every graph G with minimum degree $\delta \geq 2$ contains a cycle of length at least $\delta + 1$. Let $P = v_1, \ldots, v_k$ be a longest path in G. Let i be the minimum index in $\{1, 2, \ldots, k - 1\}$ such that $v_i v_k \in E(G)$. Since the degree of v_k is at least 2 and P is a longest (u, v)-path, such an index i exists. Moreover, the neighbors of v_k belong to $\{v_i, \ldots, v_{k-1}\}$. This implies that $k - i \geq d(v_k) \geq \delta$ and $v_i, v_{i+1}, \ldots, v_k, v_i$ is a cycle of length $k - i + 1 \geq \delta + 1$.



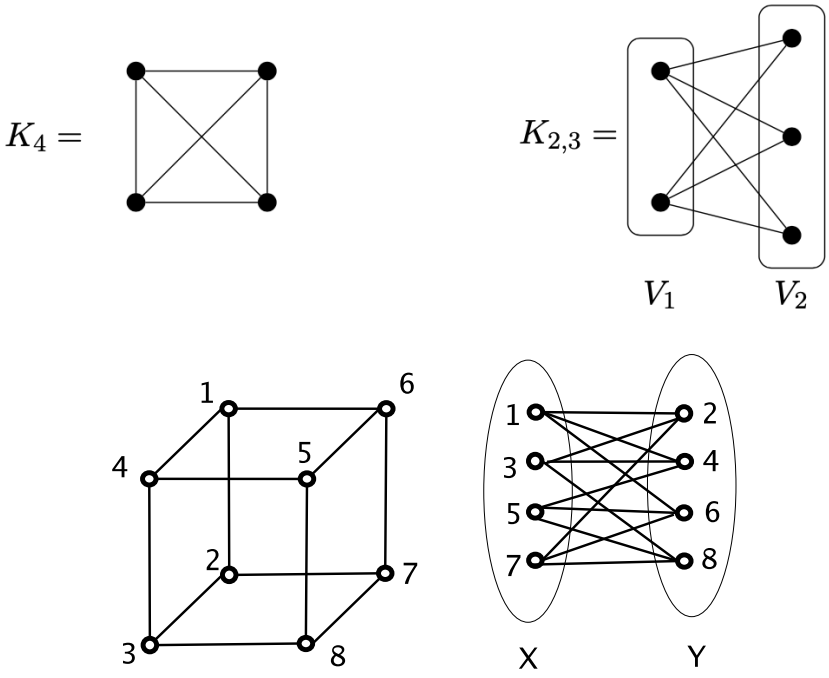
This completes the proof. □

1.7 Special graphs

- A graph where every pair of distinct vertices are adjacent, is called a **complete graph**. K_n denotes the complete graph on n vertices.
- A graph G is **bipartite** if there is a partition of $V(G)$ into two disjoint sets V_1 and V_2 such that each edge $e \in E(G)$ intersects both V_1 and

V_2. If every $v \in V_1$ and $u \in V_2$ are adjacent, then we say that G is a **complete bipartite graph**.

- $K_{n,m}$ denotes the complete bipartite graph where one side has n vertices and the other side has m vertices.



Example of a bipartite graph

2 Representation of graphs

There are many ways to represent a graph. We have already seen two ways. You can draw it (which would be difficult for large graphs), as in Figure 3 for example, or you can represent it with two sets — vertex and edge sets. In computer science, we use different methods to represent graphs – adjacency matrix, incidence matrix and adjacency list.

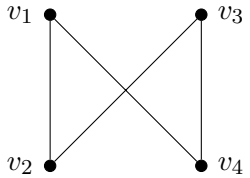
Notation: For a positive integer n, we use $[n]$ to denote the set $\{1, 2, \ldots, n\}$.

Definition 2.1 (Adjacency matrix)**.** For a graph $G = (V, E)$ with vertex set $V = \{v_1, \ldots, v_n\}$, the **adjacency matrix** of G, denoted by A_G, is the $n \times n$ matrix defined as follows.

$$\text{For every } i \in [n] \text{ and } j \in [n], \quad A_G[i,j] = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise} \end{cases}$$

Here, $A_G[i,j]$ is the entry in the ith row and jth column of A_G.

Example 2.2. Below is an example of a graph with its adjacency matrix.

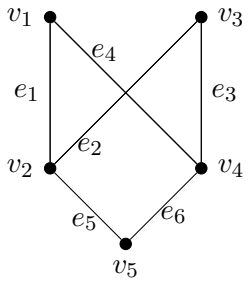


$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Definition 2.3 (Incidence matrix)**.** For a graph $G = (V, E)$ with vertex set $V = \{v_1, \ldots, v_n\}$ and edge set $E = \{e_1, \ldots, e_m\}$, the **incidence matrix** M_G of G, is the $n \times m$ matrix defined as follows.

$$\text{For every } i \in [n] \text{ and } j \in [m], \quad M_G[i,j] = \begin{cases} 1 & \text{if } v_i \in e_j \\ 0 & \text{otherwise} \end{cases}$$

Example 2.4. Below is an example of a graph with its incidence matrix.

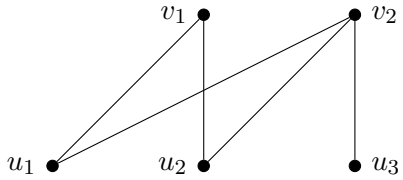


$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Definition 2.5 (Biadjacency matrix)**.** For a bipartite graph $G = (U, V, E)$ with parts $U = \{u_1, \ldots, u_r\}$ and $V = \{v_1, \ldots, v_s\}$, the **biadjacency matrix** B_G of G, is the $r \times s$ matrix defined as follows.

$$\text{For every } i \in [r] \text{ and } j \in [s], \quad B_G[i,j] = \begin{cases} 1 & \text{if } \{u_i, v_j\} \in E \\ 0 & \text{otherwise} \end{cases}$$

Example 2.6. Below is an example of a bipartite graph with its biadjacency matrix.



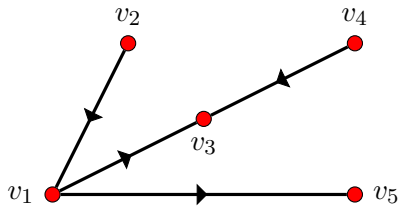
$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Definition 2.7 (Adjacency matrix of a digraph)**.** For a digraph $D = (V, A)$ with vertex set $V = \{v_1, \ldots, v_n\}$, the **adjacency matrix** of D, denoted by A_D, is the $n \times n$ matrix defined as follows.

$$\text{For every } i \in [n] \text{ and } j \in [n], \quad A_D[i,j] = \begin{cases} 1 & \text{if } (v_i, v_j) \in A \\ 0 & \text{otherwise} \end{cases}$$

Here, $A_D[i,j]$ is the entry in the ith row and jth column of A_D.

Example 2.8. Below is an example of a digraph with its adjacency matrix.

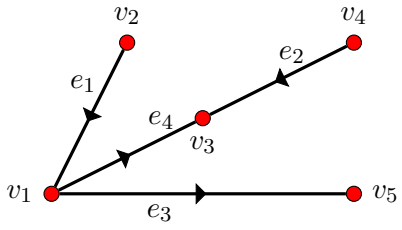


$$\begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Definition 2.9 (Incidence matrix of a digraph)**.** For a digraph $D = (V, A)$ with vertex set $V = \{v_1, \ldots, v_n\}$ and arc set $A = \{e_1, \ldots, e_m\}$, the **incidence matrix** M_D of D, is the $n \times m$ matrix defined as follows.

$$\text{For every } i \in [n] \text{ and } j \in [m], \quad M_D[i,j] = \begin{cases} 1 & \text{if } v_i \text{ is the tail of } e_j \\ -1 & \text{if } v_i \text{ is the head of } e_j \\ 0 & \text{otherwise} \end{cases}$$

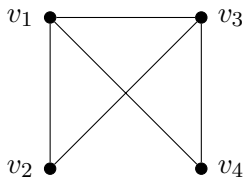
Example 2.10. Below is an example of a digraph with its incidence matrix.



$$\begin{bmatrix} -1 & 0 & +1 & +1 \\ +1 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 \\ 0 & +1 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Adjacency List. An **adjacency list** representation for a graph associates each vertex in the graph with a list of its neighbours. That is, for a graph G with vertex set $V(G) = \{v_1, \ldots, v_n\}$, we have n lists $L_{v_1}, \ldots, L_{v_n}$ where for each $i \in [n]$ the list L_{v_i} contains the neighbours of v_i in some order.

Example 2.11. Below is an example of a graph with its adjacency list.



$$\begin{aligned} L_{v_1} &:= [v_4, v_2, v_3] \\ L_{v_2} &:= [v_1, v_3] \\ L_{v_3} &:= [v_4, v_1, v_2] \\ L_{v_4} &:= [v_1, v_3] \end{aligned}$$

3 Computation on graphs

3.1 Number of Walks

We have seen how to represent graphs using matrices and lists. Let us discuss computation on graphs. As a first example, how do we compute the number of walks of length k between two vertices v_i and v_j in a graph G? First let us recall the definition of matrix multiplication.

Definition 3.1 (Matrix multiplication)**.** For an $m \times n$ matrix A and an $n \times p$ matrix B,

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, \quad B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{pmatrix}$$

the **matrix product** $C = A \cdot B$ (also denoted as $C = AB$) is defined to be the $m \times p$ matrix,

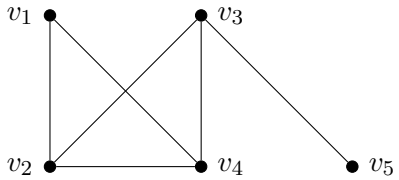
$$C = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1p} \\ c_{21} & c_{22} & \cdots & c_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mp} \end{pmatrix}$$

such that

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj} = \sum_{k=1}^{n} a_{ik}b_{kj},$$

for every $i \in \{1, \ldots, m\}$ and $j \in \{1, \ldots, p\}$

Consider the following graph G and its adjacency matrix A_G.



$$A_G = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

The matrices $A_G^2 = A_G A_G$ and $A_G^3 = A_G A_G A_G$ are given below.

$$A_G^2 = \begin{bmatrix} 2 & 1 & 2 & 1 & 0 \\ 1 & 3 & 1 & 2 & 1 \\ 2 & 1 & 3 & 1 & 0 \\ 1 & 2 & 1 & 3 & 1 \\ 0 & 1 & 0 & 1 & 1 \end{bmatrix} \qquad A_G^3 = \begin{bmatrix} 2 & 5 & 2 & 5 & 2 \\ 5 & 4 & 6 & 5 & 1 \\ 2 & 6 & 2 & 6 & 3 \\ 5 & 5 & 6 & 4 & 1 \\ 2 & 1 & 3 & 1 & 0 \end{bmatrix}$$

The value of $A_G^2[1,2] = 1$ and there is exactly one walk of length two from v_1 to v_2. This walk is v_1, v_4, v_2. The value of $A_G^3[1,2] = 5$. Indeed there are exactly 5 walks of length 3 from v_1 to v_2. They are listed below.

$$v_1, v_2, v_1, v_2$$
$$v_1, v_2, v_3, v_2$$
$$v_1, v_2, v_4, v_2$$
$$v_1, v_4, v_3, v_2$$
$$v_1, v_4, v_1, v_2$$

Also, notice that $A^3_G[2,1] = 5$, which is same as the number of walks of length 3 from v_2 to v_1. Indeed for a positive integer k, $A^k_G[i,j]$ is the number of walks of length k from v_i to v_j in G. We formally prove this in the below theorem.

Theorem 3.2. *Let G be a graph and $V(G) = \{v_1, \ldots, v_n\}$. Let A_G be the adjacency matrix of G and k be a positive intiger. Then, for every $i, j \in \{1, \ldots, n\}$, the number of walks of length k from v_i to v_j is $A^k_G[i,j]$.*

Proof. We prove the statement using induction on k.

Basis step: $k = 1$. Notice that for any $i, j \in \{1, 2, \ldots, n\}$, $A^1_G[i,j] = 1$ if and only if $\{v_i, v_j\} \in E(G)$. This implies that $A^1_G[i,j] = 1$ if and only if there is a walk of length 1 from v_i to v_j. Since G is a graph (and does not have multiedges), there will not be more than one walk of length 1 between any two vertices. This completes the basis step.

Induction step: Here, $k > 1$. By induction hypothesis (I.H) we have that for any $i, r \in \{1, 2, \ldots, n\}$, the number of walks of length $k-1$ from v_i to v_r is $A^{k-1}_G[i,r]$. Notice that

$$\begin{aligned} A^k_G[i,j] &= \sum_{r=1}^{n} A^{k-1}_G[i,r] \cdot A_G[r,j] \qquad \text{(By Def. of matrix multiplication)} \\ &= \sum_{r=1}^{n} (\text{No. of walks of length } k-1 \text{ from } v_i \text{ to } v_r) \cdot \\ &\qquad (\text{No. of walks of length 1 from } v_r \text{ to } v_j) \qquad \text{(By I.H)} \\ &= \text{No. of walks of length } k \text{ from } v_i \text{ to } v_j. \end{aligned}$$

□

Consider multiplication of two $n \times n$ real matrices. Each entry in the resultant matrix is obtained by n multiplication of real numbers and n addition of real numbers. So the total number of addition operations plus the multiplication operations in a matrix multiplication is $n^2 \cdot 2n = 2n^3$. Thus the number of steps of a naive algorithm to compute A^k_G is at most $2k \cdot n^3$, where $n = |V(G)|$.

The input to the algorithm is an $n \times n$ matrix A_G (where each entry is 0 or 1) and an integer k. So the input size is $N = n^2 + \log_2 k$. The number of steps the algorithm takes to compute A^k_G is at most $2k \cdot n^3$, Thus, when $k \leq n$, the number of steps the algorithm takes is at most $2n^4$. Thus, when

$k \leq n$, the number of steps of the algorithm is at most $2n^4 \leq N^5$. So, we say that the algorithm runs in polynomial time for $k \leq n$.

Exercise 3.3. Design an algorithm to compute A_G^k with number of steps at most $c \cdot n^3 \log_2 k$, where c is a constant (i.e., independent of n and k).

Shortest Paths Let G be a graph. For any two vertices $u, v \in V(G)$, a path (if it exists) from u to v of shortest length is called a **shortest** (u, v)**-path**. The **distance** from u to v in G, denoted by $\mathsf{dist}_G(u, v)$, is the length of a shortest path (if it exists) from u to v. If there is no (u, v)-path in G then $\mathsf{dist}_G(u, v) = \infty$. That is, dist_G is a function from $V(G) \times V(G)$ to $\mathbb{R}_{\geq 0} \cup \{\infty\}$.

Let G be a graph and let $V(G) = \{v_1, \ldots, v_n\}$. How do we compute $\mathsf{dist}_G(v_i, v_j)$? If $v_i = v_j$, then $\mathsf{dist}_G(v_i, v_j) = 0$. Otherwise, it is the smallest ℓ (less than $n = |V(G)|$) such that $A_G^\ell[i, j] > 0$. If no such ℓ exists, then $\mathsf{dist}_G(v_i, v_j) = \infty$. The above arguments are true because the shortest (v_i, v_j)-walk is a shortest (v_i, v_j)-path. We have already discussed that the computation of $A_G, A_G^2, \ldots, A_G^{n-1}$ takes polynomial time.

3.2 Graph Isomorphism

Graph Isomorphism
Input: Two graph G and H
Output: YES if G and H are isomporphic and NO otherwise.

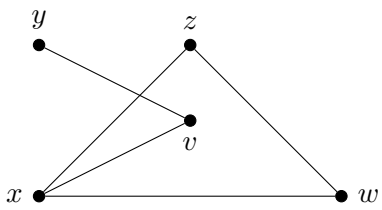
The problem is not known to be solvable in polynomial time. In 2015, László Babai announced a "quasi-polynomial time" algorithm for the problem, but retracted the claim after Harald Helfgott discovered a flaw in the proof. Babai announced a correction to the proof in 2017 with Helfgott confirming the fix. Getting a polynomial time algorithm for Graph Isomorphism is a major open problem in computer science.

4 Connectivity

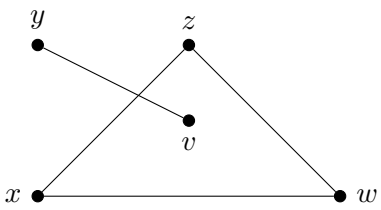
4.1 Connected components

Definition 4.1. A graph G is **connected** if for any two vertices $u, v \in V(G)$, there is a path from u to v.

Example 4.2. The graph on the left side is connected. The graph on the right side is not connected because there is no path from x to y.



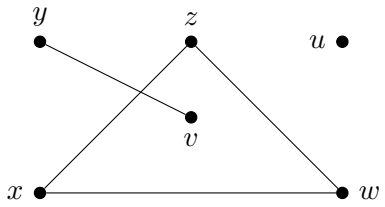
Connected graph



Disconnected graph

Definition 4.3. A **connected component** (also called **component**) of a graph G is an inclusion maximal connected (induced) subgraph of G. Here, the inclusion maximal connected subgraph C of G means that C is a connected subgraph of G, but for every subgraph C' of G such that C is a proper subgraph of C', the subgraph C' is not connected.

Example 4.4. The number of connected components in the below graph is 3. They are $C_1 = (\{w, x, z\}, \{wx, xz, zw\})$, $C_2 = (\{v, y\}, \{vy\})$, and $C_3 = (\{u\}, \emptyset)$.



Lemma 4.5. *A graph with n vertices and m edges has at least $n - m$ connected components.*

Proof. Start with the edgeless graph on n vertices, which has n components, and add edges one-by-one. Note that adding an edge can decrease the number of components by at most 1. □

4.2 Algorithm: Connectivity

Connectivity **Input:** A graph G **Output:** YES if G is connected and NO otherwise.

How to test whether a given graph G is connected? For that we may fix a vertex s and check if there is a path from s to all other vertices. This

can be tested by by checking the entries in the matrices $A_G, A_G^2, \ldots, A_G^{n-1}$. That is, for any two distinct vertices $v_i, v_j \in V(G)$, there is path from v_i to a vertex v_j in G if and only if at least one of $A_G[i,j], A_G^2[i,j], \ldots, A_G^{n-1}[i,j]$ is non-zero. This computation can be done in polynomial time. Thus, the CONNECTIVITY problem is polynomial time solvable.

5 Euler's Theorem

5.1 Eulerian tail and tour

Recall that a trail is a walk with no repeated edges.

Definition 5.1. An **Eulerian trail** in a (multi)graph G is a walk in G passing through every edge exactly once. If this walk is closed (starts and ends at the same vertex) it is called an **Eulerian tour**.

Recall the Kongsberg Bridge problem. In this problem we were looking for an Eulerian tour.

Theorem 5.2 (Euler's Theorem)**.** *A connected (multi)graph with at least one edge has an Eulerian tour if and only if each vertex has even degree.*

Proof. We prove the theorem for graphs. This proof can be easily extended to multigraphs. Let G be a connected graph with at least one edge.

($\Rightarrow$) Suppose that G has an Eulerian tour C that begins and ends at vertex v. Each time a vertex u occurs as an internal vertex of C, two of the edges incident to u are used. Since C contains every edge of G, $d(u)$ is even for every $u \neq v$. Since C starts and ends at v, $d(v)$ is even.

($\Leftarrow$) Assume that G has no vertices of odd degree. Let C be a trail in G of maximum possible length.

Claim 1: C is a closed trail.

Proof of Claim 1. (Proof by contradiction)
Suppose C is not a closed trail. Then, C has an odd number of edges (from C) incident to the final vertex v of the trail. However, as v has even degree in G, there is an edge incident to v that is not in C. This edge can be used to extend C to a longer trail, contradicting the maximality of C. □

We have proved that C is a closed trail in G. Moreover, by assumption, we have that C is a trail with maximum possible length.

Claim 2: C is an Eulerian tour.

Proof of Claim 2. (Proof by contradiction) Let $C = v_1, \ldots, v_\ell$. Since C is a closed trail, $v_1 = v_\ell$. For the sake of contradiction assume that C does not include all the edges of G. Then, since G is connected, there is an edge e outside of C such that $e = uv_i$ for some vertex v_i in C. But then $C^\star = u, v_i, v_{i+1}, \ldots, v_{\ell-1}, v_1, \ldots, v_{i-1}, v_i$ is a trail in G which is longer than C. This is a contradiction to the fact that C is a longest trail in G. Thus, we conclude that C includes all the edges of G and so it is an Eulerian tour. □

This completes the reverse direction of the proof. □

Theorem 5.3. *A connected (multi)graph G with at least one edge has an Eulerian trail if and only if it has either* 0 *or* 2 *vertices of odd degree.*

Proof. Let G be a connected (multi)graph with at least one edge.

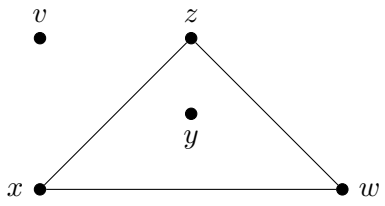
($\Rightarrow$) Suppose T is an Eulerian trail from vertex u to vertex v. If $u = v$, then T is an Eulerian tour. Then, by Theorem 5.2, all the vertices in G have even degree. Suppose $u \neq v$. Now, consider the multigraph G' obtained by adding a new edge $e = uv$ to G. Then, G' has an Eulerian tour, by using the edges in T followed by e. By Theorem 5.2, it follows that all the degrees in G' are even. Thus, we conclude that, in the original graph G, the vertices u, v are the only ones which have odd degrees.

($\Leftarrow$) If G has no vertices of odd degree then by Theorem 5.2 it contains an Eulerian tour which is also an Eulerian trail. Suppose now that G has 2 vertices u and v of odd degrees. Let G' be the graph obtained by adding an edge $e = uv$ to G. Then, all the vertices in G' have even degrees. Since G is a connected, G' is also connected. Thus, by Theorem 5.2, it has an Eulerian tour $T^\star$. Removing the edge e from $T^\star$ gives an Eulerian trail of G from u to v. □

5.2 Recognition algorithm for Eulerian graphs

Eulerian Graph Recognition
Input: A graph G
Output: YES if G is Eulerian and NO otherwise.

By Theorem 5.2 we know that any connected graph with at least one edge has an Eulerian tour if and only if every vertex has even degree. What about disconnected graphs? In fact, the below graph has an Eulerian tour. It is easy to see that a graph G has an Eulerian tour if and only if it has at most one connected component C with more than one vertex and there is an Eulerian tour in C. That is, a graph G has an Eulerian tour if and only if $G[U]$ is connected and every vertex in U has even degree, where $U = \{u \in V(G) \mid d_G(u) > 0\}$.



Algorithm for Eulerian Graph Recognition**:**

Step 1: Let $U = \{u \in V(G) \mid d_G(u) > 0\}$.

Step 2: Check whether $G[U]$ is connected.
If so, then continue; otherwise return NO and stop.

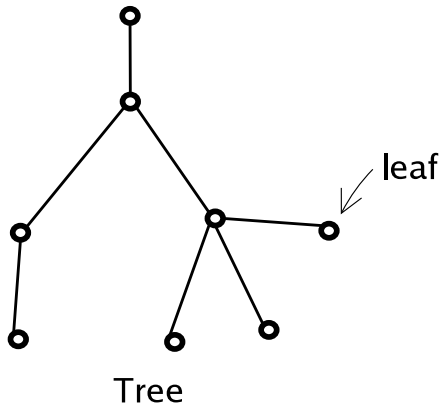
Step 3: Check whether the degree of every vertex of G is even.
If not, return NO and stop.

Step 4: Return YES;

Correctness of this algorithm follows from Euler's Theorem. Since both steps of algorithm takes polynomial time, Eulerian Graph Recognition is polynomial time solvable.

6 Trees

Definition 6.1. A graph not containing a cycle is called a **forest** (a.k.a **acyclic graph**). A **tree** is a connected acyclic graph. A leaf (or pendant vertex) in a forest is a vertex of degree 1.



Lemma 6.2. *Any tree with at least two vertices has at least two leaves.*

Proof. Let G be a tree with at least two vertices. Every connected graph with at least two vertices has an edge. Let P be a path of maximum length in G. Let u and v be the endpoints of P. Since P is of length at least 1, $u \neq v$. Suppose that u has degree greater than 1. Let u' be the neighbor of u in P, and let u'' be another neighbor of u. If u'' is in P, then there is a cycle. So u'' is not in P. Then P plus uu'' is a path of length longer than P, contradicting our assumption on P. Hence u is of degree 1, and similarly v is of degree 1. □

Deleting a leaf from an n-vertex tree produces a tree with $n-1$ vertices.

Definition 6.3 (Cut-edge)**.** In a connected graph G, an edge e is a *cut-edge* if the graph obtained from G by removing e is disconnected.

Lemma 6.4. *Let G be a connected graph. An edge e is not a cut-edge if and only if it belongs to a cycle.*

Proof. Let G be a connected graph, and let $e = xy$ be an edge of G.

($\Rightarrow$) Assume e is not a cut-edge. Let G' be the graph obtained from G by removing the edge e. Then G' is connected. So G' contains a path from x to y. But then P and e form a cycle.

($\Leftarrow$) Assume e belongs to a cycle C. Let C' be a path from x to y obtained by removing e from C. Let G' be the graph obtained from G by removing the edge e. We now show that G' is connected. Let u and v be any two vertices of G'. Since G is connected, there is a path P from u to v in G.

If P does not contain e, then P is also a path in G'. So assume P contains e. Form a walk W from u to v by deleting e from P and replacing it with C'. W is a (u, v)-walk in G'. Thus, by Lemma 1.28, there is a (u, v)-path in G'. So, for every $u, v \in V(G')$, there exists a path from u to v in G'. Therefore, G' is connected, and hence e is not a cut-edge. □

6.1 Characterization of Trees

The following theorem characterizes trees with n vertices.

Theorem 6.5. *Let $n \geq 1$ be an integer. For an n-vertex graph G, the following statements are equivalent.*

(a) G is a connected acyclic graph.

(b) G is connected and has exactly $n - 1$ edges.

(c) G is acyclic and has $n - 1$ edges.

(d) For every pair of vertices $u, v \in V(G)$, there is exactly one path from u to v in G.

Proof. First we prove (a) ⇒ (b) ⇒ (c) ⇒ (a).

Proof of (a) ⇒ (b):

Let us first clarify the statement that we want to prove. We want to prove that for every $n \geq 1$, if G is a graph on n vertices that is connected and has no cycles, then G is connected and has $n - 1$ edges.
In other words, we need to prove that for every $n \geq 1$, if G is a tree on n vertices, then G has $n - 1$ edges.

So the statement is of the form $(\forall n \geq 1)\ P(n)$, where $P(n)$ stands for 'if G is a tree on n vertices, then G has $n - 1$ edges'.
We prove this by induction on n.

Base Case (i.e. we prove P(1)):
If G is a tree (and hence a graph) on $n = 1$ vertex, it clearly has no edges (i.e. it has 0 edges).

Induction Step (i.e. we prove $(\forall n > 1)\ P(n-1) \Rightarrow P(n)$):
Let $n > 1$ and assume that $P(n-1)$ is true, i.e. assume that if G is a tree on $n-1$ vertices, then G has $n-2$ edges (this is the induction hypothesis). We need to prove that $P(n)$ is true, i.e. we need to prove that if G is a tree

on n vertices, then G has $n-1$ edges. Assume G is a tree on n vertices. By Lemma 6.2, G contains a leaf v. Let G' be the graph obtained from G by removing v (and the edge incident on v). Then G' is a tree on $n-1$ vertices. By the induction hypothesis, $|E(G')| = n-2$, and hence $|E(G)| = n-1$. This completes the proof of (a) $\Rightarrow$ (b).

For the remainder of the proof, let G be a graph with $n \geq 1$ vertices.

Proof of (b) $\Rightarrow$ (c):
Assume that G is connected and has $n-1$ edges. Delete edges from cycles of G one by one until the resulting graph G' has no cycles. By Lemma 6.4, no edge of a cycle is a cut-edge, and hence G' is connected. G' is connected and has no cycles, so by (a) $\Rightarrow$ (b), G' has $n-1$ edges. Since $|E(G)| = n-1$, it means that no edges were removed, and hence G has no cycles. This completes the proof of (b) $\Rightarrow$ (c).

Proof of (c) $\Rightarrow$ (a):
Assume that G has $n-1$ edges and no cycles. Suppose that G has k components $G_1, \ldots, G_k$. Let $n_i = |V(G_i)|$ for all $i \in \{1, \ldots, k\}$. Since G has no cycles, each component satisfies (a). So by (a) $\Rightarrow$ (b), for every $i \in \{1, \ldots, k\}$, G_i has $n_i - 1$ edges.

$$|E(G)| = \sum_{i=1}^{k} |E(G_i)| = \sum_{i=1}^{k} (n_i - 1) = n - k.$$

Since by assumption G has $n-1$ edges, it follows that $k = 1$, i.e. G is connected. This completes the proof of (c) $\Rightarrow$ (a).

So far we have shown that (a) $\Leftrightarrow$ (b) $\Leftrightarrow$ (c) (indeed, since (a) $\Rightarrow$ (b) and (b) $\Rightarrow$ (c), by transitivity of $\Rightarrow$ it follows that (a) $\Rightarrow$ (c), and since (a) $\Rightarrow$ (c) and (c) $\Rightarrow$ (a), it follows that (a) $\Leftrightarrow$ (c); the other equivalences follow similarly). To complete the proof, we now show that (a) $\Leftrightarrow$ (d).

Proof of (a) $\Rightarrow$ (d):
Let G be a tree. Let u and v be any two vertices of G. Since G is connected, it follows that there is at least one path between any pair of vertices u and v. (We now argue by contradiction). Suppose that there are 2 distinct paths from u to v, P_1 and P_2. Since P_1 and P_2 are distinct, there is an edge $e = xy$ that is in P_1 but not in P_2. Let G' be the graph obtained by putting

together nodes and edges of P_1 and P_2 and deleting the edge e. G' contains a walk P from x to y, and hence by Lemma 1.28 it contains a path P from x to y. But then P together with edge e is a cycle, a contradiction.
This completes the proof of (a) $\Rightarrow$ (d).

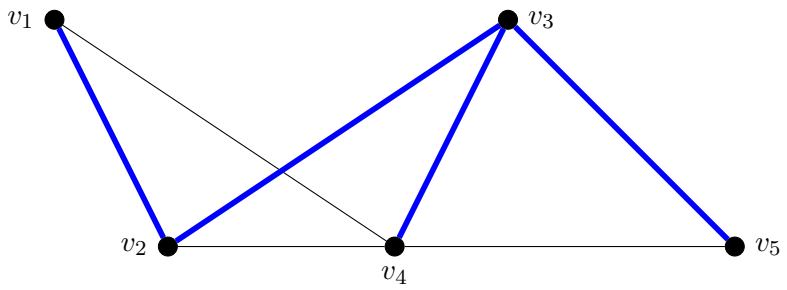
Proof of (d) $\Rightarrow$ (a):
Suppose that any 2 vertices of G are connected by a unique path. Then G is connected. Next we prove by contradiction that G is acyclic. Suppose G contains a cycle C. Then, C contains at least two vertices, and hence for distinct vertices x and y of C, there exist distinct paths from x to y in C, one obtained by traversing C from x to y in one direction and the other obtained by traversing C from x to y in the other direction. Note that since C is a cycle, these two paths do not have any edges in common. This is a contradiction. This completes the proof of (d) $\Rightarrow$ (a). □

How to test a given graph is a tree? Our algorithm needs to check if the given graph G a connected and $|E(G)| = |V(G)| - 1$.

6.2 Spanning Trees

Definition 6.6. A **spanning tree** T of a connected graph G is a spanning subgraph of G and it is a tree. That is, T is a subgraph of G, T is a tree, and $V(T) = V(G)$.

Example 6.7. A graph where the edges of a spanning tree are thickened and colored blue.



How can we compute a spanning tree of a graph G. In fact, by Theorem 6.5, we know that a graph T is a tree if T is connected and $|E(T)| = |V(T)| - 1$. We can use this fact to keep deleting edges from G, maintaining connectivity until we reach a tree.

Spanning Tree
Input: A graph G.
Output: A spanning tree of G if it is connected. Otherwise output NO

A pseudocode for solving Spanning Tree is given in Algorithm 1. The correctness of it follows from Theorem 6.5.

Algorithm 1 Algorithm for Spanning Tree

```
1: if G is not connected then
2:     Output NO.
3: end if
4: while |E(G)| > |V(G)| − 1 do
5:     Pick an edge e such that G − e is connected.
6:     G := G − e                                   (i.e., delete e from G)
7: end while
8: Output G.
```

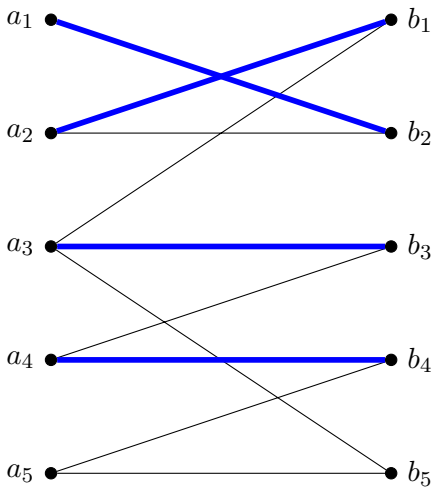
Informal discussion about running time. Let $n = |V(G)|$ and $m = |E(G)| \leq n^2$. The **While** loop executes at most $m - n + 1$ iterations. In each iteration we use the procedure for the Connectivity problem, which runs in polynomial time (say n^c for a constant c). So the total cost of the execution of the **While** loop for all iterations is at most $n^{c+2} + m \leq 2n^{c+2}$. Again Step 1 take n^c time. So the total runtime is at most $2n^{c+2} + n^c + 1 \leq 4n^{c+2}$. This is a polynomial running time because $c + 2$ is a constant (independent of n).

7 Bipartite Matching and Organ Donation

7.1 Matching

Definition 7.1. A **matching** in a graph is a set M of non-overlapping edges. The vertices incident to the edges of M are **saturated** by M and the others are **unsaturated** by M. A matching of maximum cardinality in a graph is called a **maximum matching**. A **perfect matching** in a graph is a matching that saturates every vertex.

Example 7.2. A graph where the edges of a matching are thickened and colored blue. Here, $M = \{a_1b_2, a_2b_1, a_3b_3, a_4b_4\}$ is a matching. The vertices saturated by M are $a_1, a_2, a_3, a_4, b_1, b_2, b_3, b_4$. The vertices unsaturated by M are a_5 and b_5.



Notice that matchings are defined on any graph, but in this section we focus on matchings on bipartite graphs. Following are two interesting algorithmic questions on bipartite matching.

Bipartite Perfect Matching
Input: A bipartite graph G.
Output: A perfect matching in G, if it exists. Otherwise output NO.

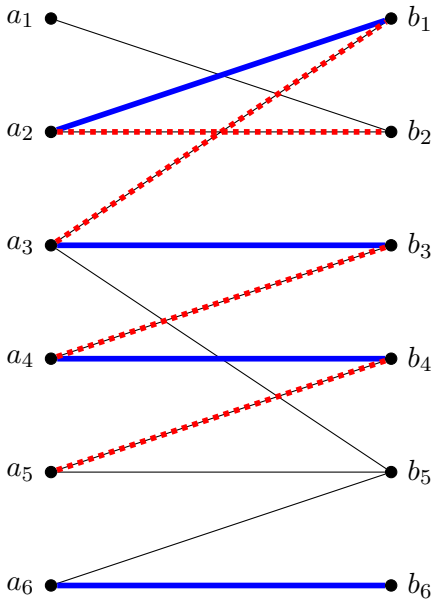
Bipartite Maximum Matching
Input: A bipartite graph G.
Output: A maximum matching in G.

7.2 Algorithm for Bipartite Maximum Matching

First let us define some terms related to matching. Let G be a bipartite graph with bipartition $V(G) = A \cup B$ and M be a matching in G.

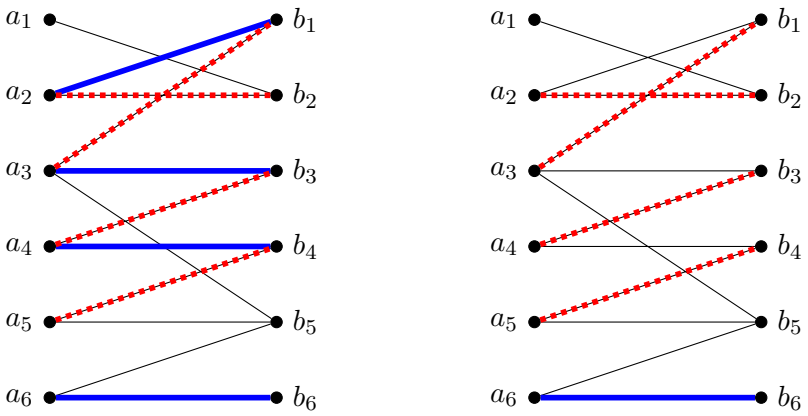
- A path P in G is called an **alternating path** with respect to M, if for every pair of subsequent edges on P, one of them is in M and another one is not in M.
- A path P in G is an **augmenting path** with respect to M, if P is an alternating path with respect to M, with an additional property that its start vertex and end vertex are unsaturated by M.

Example 7.3. Consider the bipartite graph given below. Here, $M = \{a_2b_1, a_3b_3, a_4b_4, a_6b_6\}$ is a matching. $P = a_5, b_4, a_4, b_3, a_3, b_1, a_2, b_2$ is an augmenting path with respect to M. $P' = b_4, a_4, b_3, a_3, b_1, a_2, b_2$ is an alternating path, but not an augmenting path with respect to M.



Let G be a bipartite graph and let M be a matching in G. Let P be an alternating path. Let $E(P)$ denotes the set edges in P. Then, the symmetric difference $E(P)\triangle M$ is indeed a matching. Moreover, if P is an augmenting path, then $E(P)\triangle M$ is a larger matching than M (In fact, $|E(P)\triangle M| = |M| + 1$).

Example 7.4. Consider the bipartite graph given below. Here, $M = \{a_2b_1, a_3b_3, a_4b_4, a_6b_6\}$ is a matching of cardinality 4. $P = b_2, a_2, b_1, a_3, b_3, a_4, b_4, a_5$ is an augmenting path with respect to M. $E(P) \setminus M = \{a_2b_2, a_3b_1, a_4b_3, a_5b_4, a_6b_6\}$ is a matching of cardinality 5.



Exercise 7.5. Let G be a bipartite graph and let M be a matching. Let P be an augmenting path with respect to M. Then, prove that P is of odd length.

Lemma 7.6. *Let G be a bipartite graph and M be a matching in G. Then, M is a maximum matching if and only if there is no augmenting path with respect to M.*

Proof. ($\Rightarrow$) We prove the contrapositive statement. That is, we prove that if G has an augmenting path with respect to M, then M is not a maximum matching. Let P be an augmenting path with respect to M. Then $E(P) \setminus M$ is a matching of cardinality $|M|+1$. Hence, M is not a maximum matching.

($\Leftarrow$) We prove this direction also by contrapositive. That is, we prove that if M is not a maximum matching, then G has an augmenting path with respect to M. Let $M^\star$ be a maximum matching. We know that $|M^\star| > |M|$. Consider the symmetric difference $M^\star \triangle M$ of $M^\star$ and M. Let $S = M^\star \triangle M$. As both M and $M^\star$ are matchings, each vertex can be incident to at most 2 edges in S, because it can be incident to at most one edge from each matching. If a vertex is incident to 2 edges in S, then it is incident to one edge from M and one edge from $M^\star$. Now consider the subgraph H with vertex set $V(G)$ and edge set S. Each vertex in S has degree at most 2 and each connected component in H is one of the following.

- A single vertex.
- A path whose edges are alternately from M and $M^\star$.
- A cycle of even length whose edges are alternately from M and $M^\star$.

Any connected component of H which forms a cycle will contain exactly same number of edges from both $M^\star$ and M. Since $|M^\star| > |M|$, S has more edges from $M^\star$ than M. Thus, at least one connected component of H will be a path P having more edges from $M^\star$ than M. Because any such path is alternating, it will have start and end vertices unsaturated by M. That is, P is an augmenting path with respect to M. □

The above lemma can be used to design an algorithm for Bipartite Maximum Matching. See Algorithm 2.

Algorithm 2 Algorithm for Bipartite Maximum Matching

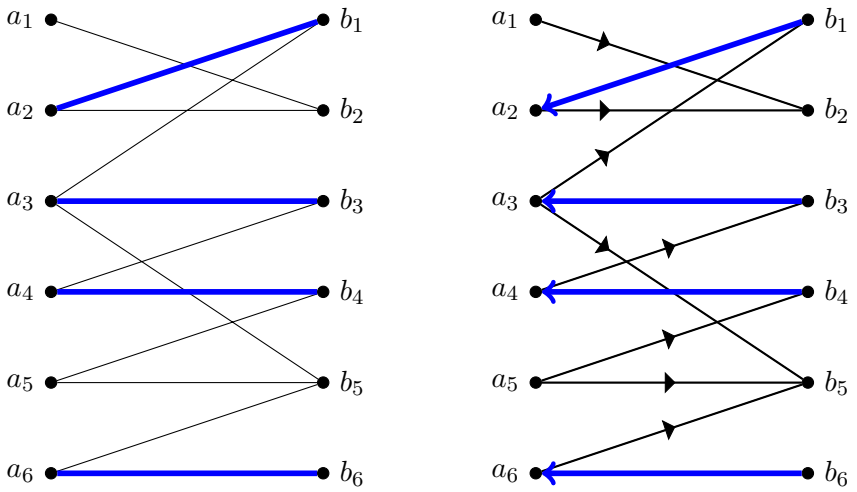
```
1: M := ∅.
2: while there is an augmenting P path w.r.t M do
3:     M := E(P)△M
4: end while
5: Output M.
```

But, the crucial step in Algorithm 2 is how to compute an augmenting path P with respect to M. Computation of such a path can be reduced to the problem of finding a (shortest) path in a digraph as follows. Let G be a bipartite graph with bipartition $V(G) = A \cup B$ and M be a matching. Construct a digraph D from G where each edge is directed as follows. An edge $\{a, b\} \in M$ is directed from b to a. An edge $\{a', b'\} \in E(G) \setminus M$ is directed from a' to b'. See Example 7.7 for an illustration.

Then, any directed path in D from an unsaturated vertex in A to an unsaturated vertex in B is an augmenting path with respect to M in G. If G has an augmenting path with respect to M, then G has an augmenting path from an unsaturated vertex in A to an unsaturated vertex in B. Why is this true? In Exercise 7.5, we proved that every augmenting path has odd length. So any augmenting path has one end vertex in A and the other in B. So for any augmenting path P in G, either P or the reverse of P, is starting from A and ending at B. If P' is an augmenting path in G from a vertex in A to a vertex in B, then P' is a directed path in D from an unsaturated vertex in A to an unsaturated vertex in B.

Thus, one can use the ideas of Single Source Shortest Paths on graphs, to develop such an algorithm on digraphs. This algorithm can be used to find an augmenting path (if it exists).

Example 7.7. Consider the bipartite graph G (picture in the left side) and a matching $M = \{a_2b_1, a_3b_3, a_4b_4, a_6b_6\}$. The digraph constructed from G is in the right side.



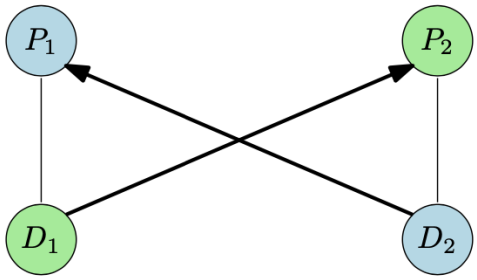
7.3 Kidney Exchange

People who suffer from kidney failure, need kidney transplant. An old idea, used also for other organs, is deceased donors. When a registered organ donor dies, their organs can be transplanted into others. One special feature of kidneys is that a healthy person has two of them and can survive just fine with only one of them. This creates the possibility of living organ donors, such as a family member of the patient in need.

Unfortunately, having a living kidney donor is not always enough. The patient-donor pair should be *compatible*. Some examples are blood and tissue types should be compatible. For example, a patient with O blood type can only receive a kidney from a donor with the same blood type, and similarly an AB donor can only donate to an AB patient.

- Type O donor can donate to Type O, A, B, AB.
- Type A donor can donate to Type A, AB.
- Type B donor can donate to Type B, AB.
- Type AB donor can donate to Type AB.

Suppose patient P_1 is *incompatible* with their donor D_1 because they have blood types A and B, respectively. Suppose P_2 and D_2 are also incompatible with blood types B and A. Then, exchanging donors is a good idea. P_1 can get kidney from D_2 and P_2 can get a kidney from from D_1. This is called a *kidney exchange*.



(Figure from [3])

In the last two decades a lot of research has been carried out for designing good algorithms to facilitate kidney exchange and to help more patients. This is an active area of research in algorithmic game theory.

Algorithmic Ideas. One algorithm to facilitate more kidney exchanges is to use matchings on graphs. Construct a graph G where each vertex is an incompatible patient donor pair (P_i, D_i). Two vertices (P_i, D_i) and (P_j, D_j) have an edge if and only if P_i and D_j are compatible, and P_j and D_j are compatible. Here, we want to find a maximum cardinality matching in G to arrange as many compatible kidney transplants as possible.

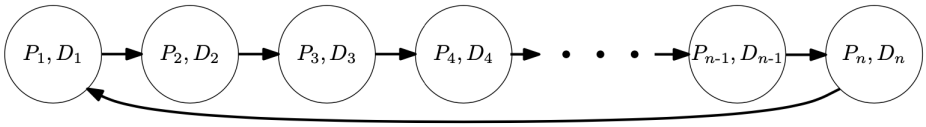
By restricting the *feasible set* to matchings of this graph, we are restricting to pairwise kidney exchanges only.

Consider the scenario, where there are three patient donor pairs (P_1, D_1), (P_2, D_2), and (P_3, D_3). The compatible pairs are the following.

- D_1 and P_2
- D_2 and P_3
- D_3 and P_1.

In this scenario our graph G is edgeless and has no solution. This motivates the idea of finding cycles in a digraph D constructed as follows.

D has a vertex for each incompatible patient donor pair (P_i, D_i). There is a directed edge from (P_i, D_i) to (P_j, D_j) if and only if D_i and P_j are compatible. In this graph we would like to find cycles covering maximum number of vertices. But in the case of kidney exchange large cycles creates a problem. Suppose $(P_1, D_1), (P_2, D_2), \ldots, (P_n, D_n), (P_1, D_1)$ is a cycle.



(Figure from [3])

Then all the $2n$ surgeries should happen simultaneously. Why? Suppose if the surgeries for D_1 and P_2 happen before the surgeries of D_2 and P_3, then there is a risk that D_2 will renege on its offer to donate its kidney to P_3 and thus the entire cycle breaks with just one kidney transplant. So if there is a large cycle in our solution, then simultaneous surgeries are not possible. Thus, one possibility is to find short cycles covering maximum number of vertices. Here, the length of the cycle depends on the number of simultaneous surgeries possible.